

Network and Computer Security — Lecture 11

Dr. Karim Lounis

Jan - Jun 2026



Part 3

Network Security

Security Protocols: The Formal View

Security Protocols

Definition

Security Protocol consists of a sequence of steps that need to be executed in a particular order to satisfy desired security properties.

- A protocol, in general, is a set of interaction rules designed to guarantee functionality (e.g., TCP: init, data exchange, and finish).
- A security protocol is a set of interaction rules designed to guarantee specific security properties as well as intended functionality.
- In practice, security protocols use existing cryptographic primitives to guarantee the desired security properties.
- Example of everyday security protocols includes, TLS, SSL, IPsec, SSH, PGP, DNSsec, MACsec, WPA2, WPA3, etc.
- We verify security protocols against the claimed security properties using formal methods, i.e., using mathematical and logical tools.

Security Protocols — The long standing Issue

“Security protocols are three line programs that people still manage to get wrong” — **Roger Needham.**

A protocol might literally be three lines of mathematical notation expressing a trivial logic:

- ① Massoud $\rightarrow$ Zoubir: “I am Massoud”
- ② Zoubir $\rightarrow$ Massoud: “Prove it”
- ③ Massoud $\rightarrow$ Zoubir: proof.

So, why people still get security protocols wrong?

Security Protocols — The long standing Issue

So, why people still get them wrong?

- **Adversarial thinking is non-intuitive.** The logic may hide massive complexity, i.e., the designer did not take into account all the possible ways to read those three lines.
- **No “mostly secure”.** Security is not gradual, i.e., small mistake (weak randomness, missing nonce, etc) is total failure.
- **Hidden Assumption.** Protocol often silently assume things (e.g., trusted channel). If the assumption is wrong, the protocol collapse.
- **Ambiguous Protocol Specification.** If specification sees non-unique interpretations, the world sees different implementations.
- **Composition is Hard.** Incorrectly combining cryptographic primitives introduce vulnerabilities.

Security Protocols — The long standing Issue

“Security protocols are three line programs that people still manage to get wrong” — **Roger Needham**.

Security is not about writing code—it's about reasoning under adversarial conditions.

✈ Needham's quote serves as a warning to engineers: Never underestimate the complexity of a simple exchange.

For example, the Needham-Schroeder Authentication Protocol (1978) was believed to be secure for 17 years until Gavin Lowe discovered a flaw using automated formal methods.

Security Protocols — The long standing Issue

“Security protocols are three line programs that people still manage to get wrong” — **Roger Needham**.

Security protocols are hard to get right. Sources of errors include:

- Insecure protocols (i.e., unsecure by design).
- Incorrect cryptographic primitives (i.e., got broken).
- Bad implementations (i.e., secure, but turned void).
- Changed assumptions on environment (i.e., threat model, CPUs).

Security Protocols — The long standing Issue

In practice, security protocols are analyzed under certain assumptions:

- ① **Perfect Cryptography.** We assume that cryptographic primitives (e.g., hash function, etc) are mathematically unbreakable.
- ② **Black-box Abstraction.** We treat the implementation of the cryptographic primitives as an “idealized black box.”.
- ③ **Environment.** Threat model— Dolev-Yao Model.

Security Protocols — The long standing Issue

In practice, security protocols are analyzed under certain assumptions:

Dolev-Yao Model:

Is an attacker model that is used when analyzing security protocols. It assumes:

- Attacker learns every messages sent over the network.
- Attacker can block messages.
- Attacker can insert messages.
- Attacker can modify messages.

Security Misconceptions (Why it's hard to get it right?)

Some misconceptions with respect to security:

- “Our application is secure because we are using the most up-to-date cryptographic algorithms.”
- “We keep our communication protocol secret, so nobody will ever be able to attack it and we don’t have to worry about security of the protocol.”
- “My engineers looked at the protocol for a while and could not find errors, so we can safely use it.”
- “Now that we have finished the design and specification of our system, we can start adding security features.”
- “In order to be sure, we apply double encryption.”
- “We have found this security protocol in literature, so we can safely embed it in our system.”
- “A friend of mine is using it, they have never been hacked.”

Security Misconceptions (Why it's hard to get it right?)

Some misconceptions with respect to security:



Cryptography is like a lock. No matter how good a lock is, if it is used incorrectly, it doesn't protect.

Security Protocol Analysis

The security of a protocol can be analyzed using two different methods:

- **Manually.** Generally, it goes as follows:
 - We model the protocol (agents and their actions) using formal languages, e.g., process algebra, operational semantics, ...
 - Specify the desired security goals the protocol should achieve. We write mathematical expressions to represent the properties.
 - Define the capabilities of a potential attacker. A common model is the Dolev-Yao attacker, who can eavesdrop, intercept, and modify messages but cannot break the cryptography itself.
 - Using the model and attacker definition, you manually analyze if the protocol achieves the desired security properties.
- **Automatically.** We can use formal verification tools, such **Tamarin**.
 - The protocol is described in a formal language the tool understands.
 - The desired security goals are defined within the tool's framework.
 - Automated Analysis: The tool performs a symbolic analysis, exploring all possible message exchanges and attacker actions. It checks if these actions violate the security properties.

Security Protocol Analysis

The security of a protocol can be analyzed using two different methods:

- **Manually.** Generally, it goes as follows:

We can use formal inference and induction systems like **BAN logic** (Burrows-Abadi-Needham logic), **Paulson's Inductive method**, **Descriptive logic**, ... **Temporal logics** (e.g., LTL, CTL, CTL*), **Process Algebra** (π -Calculus), **Second-order logic**, **Higher-order logic** or even **First order logic** to model protocols' steps and derive conclusions.

Tools	Logical Methods
Tamarin	Multiset Rewriting + LTL + FOL + Symbolic Model
AVISPA	Model Checking (HPSL)
ProVerif	Applied π -Calculus + FOL
Isabelle/HOL	Higher-Order Logic
Scyther	Model Checking (Pattern Matching)

ing all possible message exchanges and attacker actions. It checks if these actions violate the security properties.

Security Protocol Analysis (Tamarin)

Example. Consider the following protocol (mini_sec_protocol), where

Alice (A) wants to share a secret message m with Bob (B) using a three-pass protocol that was assumed to be secure:

- $A \rightarrow B : \{A, n_A\},$
- $B \rightarrow A : \{B, n_B\},$
- $A \rightarrow B : \{A, n_A \oplus n_B \oplus m\}.$

If we want to automatically analyze the security of the protocol, we can use **Tamarin**.

We write the protocol specification using Tamarin language and then push the Tamarin engine to analyze the protocol's security.

```

mini_sec_protocol.spthy
Edited
Haskell

equations:
  XOR(XOR(x, y), y) = x // Cancellations
  //XOR(x, y) = XOR(y, x) // Commutativity
  //XOR(x, XOR(y, z)) = XOR(XOR(x, y), z) // Associativity

/* Protocol rules of facts */
// Alice sends her nonce n_a to Bob
rule Alice_sends_nonce:
  [ Fr(~na) ]
  --[ SendNA(~na) ]-->
  [ Out(~na) ]

// Bob receives Alice's nonce and sends his own nonce n_b
rule Bob_sends_nonce:
  [ In(na), Fr(~nb) ]
  --[ RecvNA(na), SendNB(na, ~nb) ]-->
  [ Out(~na, ~nb) ]

// Alice receives Bob's nonce and sends the encrypted message
rule Alice_sends_message:
  [ In(~na, nb), !Secret(m) ]
  --[ RecvNB(na, nb), SendEncrypted(na, nb, m) ]-->
  [ Out(XOR(na, XOR(na, nb))) ]

// Bob receives the encrypted message and decrypts it
rule Bob_receives_message:
  [ In(encrypted), !BobNonces(na, nb) ]
  --[ RecvEncrypted(na, nb, XOR(encrypted, XOR(na, nb))) ]-->
  [ Out(XOR(encrypted, XOR(na, nb))) ]

/* Restriction facts */
rule Remember_Bob_nonces:
  [ !SendNB(na, nb) ]
  -->
  [ !BobNonces(na, nb) ]

/* Initial setup */
rule Generate_secret:
  [ Fr(~m) ]
  --[ Secret(~m) ]-->
  [ !Secret(~m) ]

/* Lemmas */
lemma message_secrecy: "All m #l. Secret(m) @i ==> not (Ex #j. K(m) @j)"
lemma authentication: "All na nb #l. RecvEncrypted(na, nb, m) @i ==>
  (Ex #j. SendEncrypted(na, nb, m) @j)"
lemma protocol_executable: exists-trace "Ex m #l. Secret(m) @i"
end

```

Security Protocol Analysis (Tamarin)

Example. Consider the following protocol (mini_sec_protocol), where Alice (*A*) wants to share a secret message *m* with Bob (*B*) using a three-pass protocol that was assumed to be secure:

Running Tamarin 1.10.0
Index Download Actions »

Proof scripts

theory MiniSec begin

Message theory

Multiset rewriting rules (8)

Tactic(s)

Raw sources (7 cases, deconstructions complete)

Refined sources (7 cases, deconstructions complete)

```

lemma message_secret:
  all-traces
  "V m #i. (Secret( m ) @ #i) → (¬(∃ #j. K( m ) @ #j)))"
simplify
solve( !KU( ~m ) @ #vk )
case Alice_sends_message
solve( splitEq(1) )
case split_case_1
solve( !KU( XOR( na, nb ) ) @ #vk.4 )
case c_XOR
  SOLVED // trace found
qed
qed
qed

lemma authentication:
  all-traces
  "V na nb m #i.
    (RecvEncrypted( na, nb, m ) @ #i) →
    (∃ #j. SendEncrypted( na, nb, m ) @ #j)"
  by sorry

```

Visualization display

Constraint System is Solved

Constraint system

last: none

Security Protocol Analysis

The security of a protocol can be analyzed using two different methods:

- **Manually.** Generally, it goes as follows:
 - We model the protocol (agents and their actions) using **formal languages**, e.g., process algebra, operational semantics, ...

We can also use **MSCs** visually analyze the security of small protocols and find traces (execution scenarios) that compromise the security claims

- Using the model and attacker definition, you manually analyze if the protocol achieves the desired security properties.
- **Automatically.** We can use formal verification tools, such **Tamarin**.
 - The protocol is described in a formal language the tool understands.
 - The desired security goals are defined within the tool's framework.
 - Automated Analysis: The tool performs a symbolic analysis, exploring all possible message exchanges and attacker actions. It checks if these actions violate the security properties.

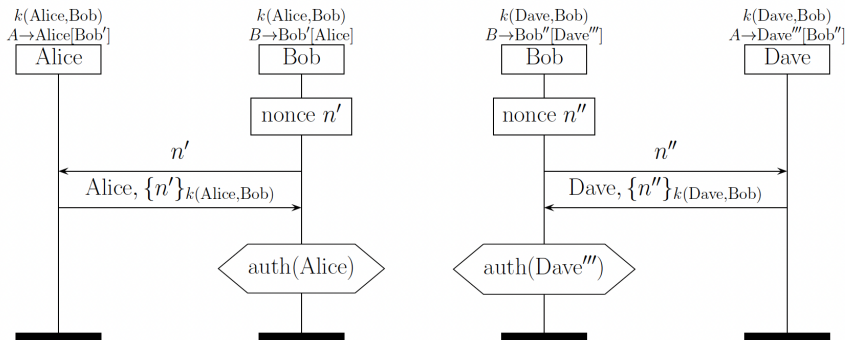
Security Protocol (Message Sequence Chart)

An **MSC** is a graphical language for the describing the interaction between different components of a system. This language is standardized by the ITU.

- Agents (parties) are represented with rectangles on top of a vertical time-line (a line during which a given agent (party) is active).
- Established security goals are represented by hexagones. Those are called “security claims”, i.e., “authenticated”, “ K secret”, ... etc.
- Operations performed by a given party (e.g., “Generate nonce n ”) are represented by a rectangle within the party’s timeline.
- The end of the execution of a communication party is represented by a **bold** horizontal line on its timeline.
- Nonces are denoted as: $n, n_1, n_2, \dots, n_A, n_B, \dots$
- Messages are denoted as: $m_1, \dots, m_n, \dots, m', m'', \dots$
- Encrypted nonce using key K is denoted: $\{n\}_K$.
- Encrypted nonce n using Zoubir’s public key: $\{n\}_{pk(Z)}$.
- Encrypted nonce n using Zoubir’s private (secret) key: $\{n\}_{sk(Z)}$.

Security Protocol (Message Sequence Chart)

The following is an MSC challenge-response protocol:

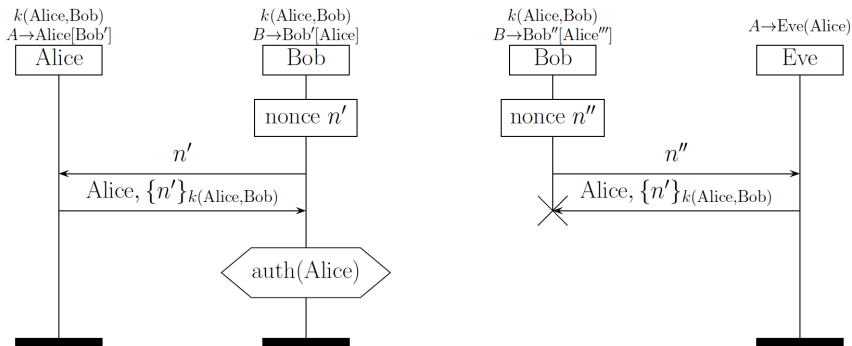


Here, Alice and Dave are authenticating themselves to Bob.

The notation $B \rightarrow \text{Bob}''[\text{Dave}''']$ means that Bob is executing the role of party B in Run 2, and Dave is executing the role of party A in Run 3.

Security Protocol (Message Sequence Chart)

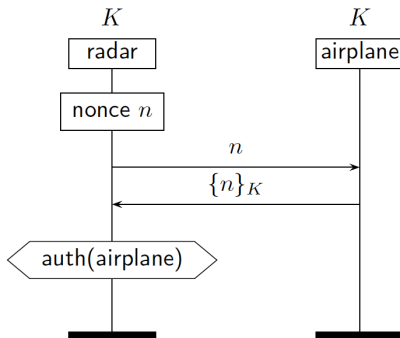
This is an unsuccessful **replay attack** on the challenge-response proto:



Eve is trying to impersonate Alice. During a previous run of the protocol, Eve captured the parameter $\{n'\}_{k(\text{Alice}, \text{Bob})}$, and tried to re-use it again.

Security Protocol (Attacking Protocols)

Example 1. Consider the following MSC for a CRP:

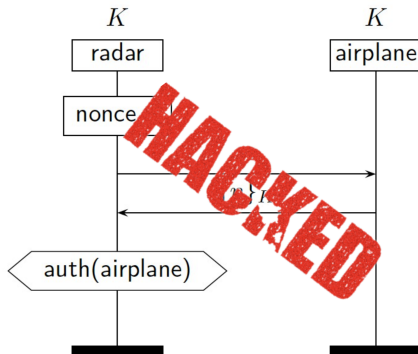


The intention is that only airplane knowing the key K is able to provide the correct response to a randomly generated and unknown in advance value n .

Security goals: Authentication, secrecy, ... (Trying to enforce)

Security Protocol (Attacking Protocols)

Example 1. Consider the following MSC for a CRP:



The intention is that only airplane knowing the key K is able to provide the correct response to a randomly generated and unknown in advance value n .

Security goals: Authentication, secrecy, ... (Trying to enforce)

Security Protocol (Attacking Protocols)

Distance Bounding Protocols. Are security protocols that in addition to trying to satisfy security properties (security claims), they check the physical distance between the involved communication parties.

$$d = \frac{\text{RTT} \times c}{2}$$

Trying to mitigate attacks such as:

- Distance fraud attack.
- Mafia fraud attack (a.k.a., relay attack).
- Terrorist fraud attack.

Applications where you have to be cautious:

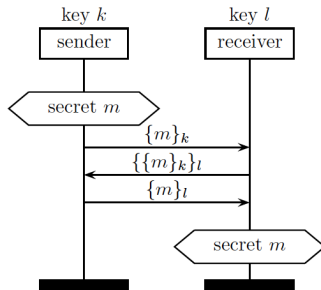
- Contactless payments (NFC).
- Keyless car entry systems.
- Secure access control using RFID cards.

Shamir's no-key protocol (Attacking Protocols)

Example 2. Recall the Shamir's no-key protocol:



a

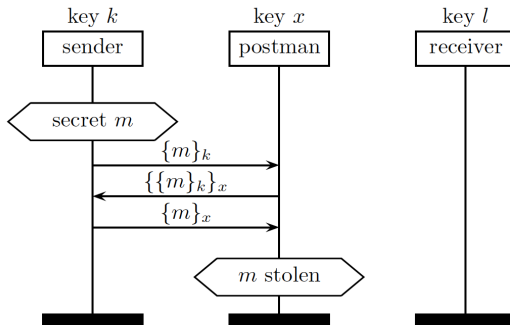


b

Trying to share a secret m without pre-sharing a secret key. This can be done using commutative crypto-systems.

Shamir's no-key protocol (Attacking Protocols)

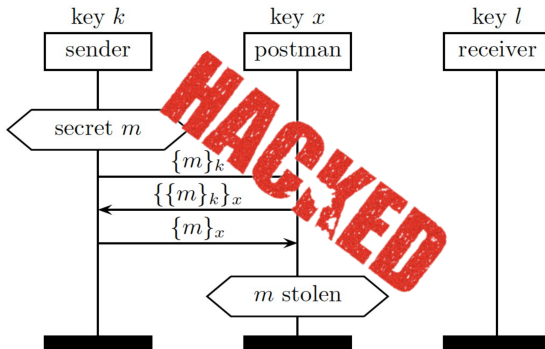
The postman can pose as a MITM and steal the secret m :



It seems that it is not possible to exchange secrets with a specific agent without already having established a shared secret or secret channel.

Shamir's no-key protocol (Attacking Protocols)

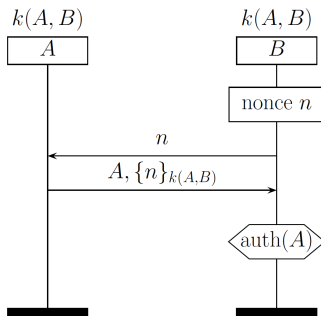
The postman can pose as a MITM and steal the secret m :



It seems that it is not possible to exchange secrets with a specific agent without already having established a shared secret or secret channel.

Security Protocol (Attacking Protocols)

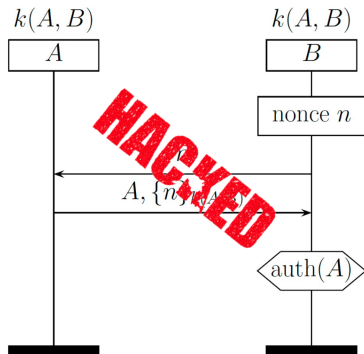
Example 3. Consider the following CR protocol:



Agent “B” decrypts $\{n\}_{k(A,B)}$ and if it finds the value n , it authenticates the identity “A”. Could you find an execution of this protocol where the authentication claim gets compromised? Propose a fix.

Security Protocol (Attacking Protocols)

Example 3. Consider the following CR protocol:



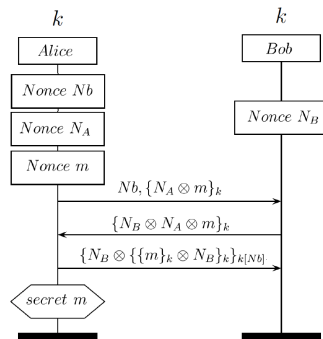
Agent “B” decrypts $\{n\}_{k(A, B)}$ and if it finds the value n , it authenticates the identity “A”. Could you find an execution of this protocol where the authentication claim gets compromised? Propose a fix.

Security Protocol (Attacking Protocols)

Example 4. Alice and Bob execute the protocol shown below to share a secret m . Assuming that the protocol uses a shared symmetric key ($k = k^{-1}$), i.e., already shared between Alice and Bob, and the encryption algorithm is the same as the decryption algorithm, and $N_b \geq 1$:

Find an attack scenario where Charlie can disclose m considering the following:

- Consider the Dolev-Yaho attack model for Charlie.
- Consider the case where parameters are incorrectly chosen.
- Hint: Charlie is just an eavesdropper.
- Propose a fix for the flaw.

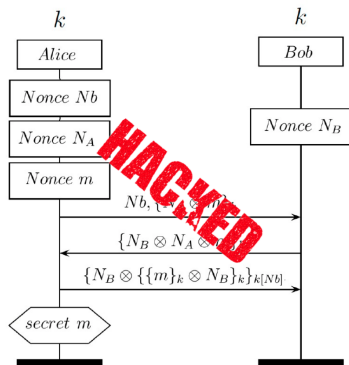


Security Protocol (Attacking Protocols)

Example 4. Alice and Bob execute the protocol shown below to share a secret m . Assuming that the protocol uses a shared symmetric key ($k = k^{-1}$), i.e., already shared between Alice and Bob, and the encryption algorithm is the same as the decryption algorithm, and $N_b \geq 1$:

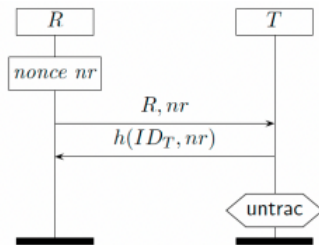
Find an attack scenario where Charlie can disclose m considering the following:

- Consider the Dolev-Yaho attacke model for Charlie.
- Consider the case where parameters are incorrectly chosen.
- Hint: Charlie is just an eavesdropper.
- Propose a fix for the flaw.



Security Protocol (Attacking Protocols)

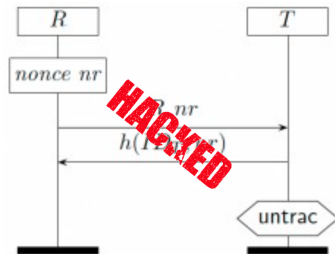
Example 5. In RFID protocols, readers R interrogate RFID tags T for identification (and, sometimes authentication).



Is the tag traceable? (i.e., we can compromise untraceability)

Security Protocol (Attacking Protocols)

Example 5. In RFID protocols, readers R interrogate RFID tags T for identification (and, sometimes authentication).



Is the tag traceable? (i.e., we can compromise untraceability)

Security Protocols: The Practical View

TLS: Transport Layer Security

Transport Layer Security (v1.0, v1.1, v1.2, and v1.3)

Definition

TLS. Is a hybrid cryptographic protocol designed to provide secure communication over a computer network. It allows application-level protocols to securely exchange data over an unsecure channel.

- The protocol aims to provide security services: authentication, confidentiality, integrity, and non-repudiation, using cryptography.
- Application-level protocols become secure over TLS: HTTP→HTTPS, SMTP→SMTPS, POP3→POP3S, DNS→DNSS, FTP→FTPS, etc.
- Communication ports change as well: 80→443, 25→587, 110→995, 53→853, 21→22, 67→68, 143→993, etc.
- TLS was first defined in 1999 (RFC2246) and the current version is TLS 1.3 (2018 — RFC8446). TLS deprecated an older protocol, called SSL (Secure Sockets Layer) — 1994 - 1996, which was developed by Netscape.
- Based on the used version, TLS allows two parties, in a number of roundtrips, to establish a shared secret key for secure communication.

Transport Layer Security (v1.0, v1.1, v1.2, and v1.3)

Cipher Suite: A cipher suite is a combination of cryptographic algorithms used to secure the communication between two parties in a TLS (Transport Layer Security) connection.

TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA256

- ECDHE a key establishment protocol.
- ECDSA signature algo to bind ID-handshake (**not in 1.3**).
- AES_256_GCM the encryption algo (**after TLS handshake**).
- SHA256 the hash function algorithm for MAC generation.

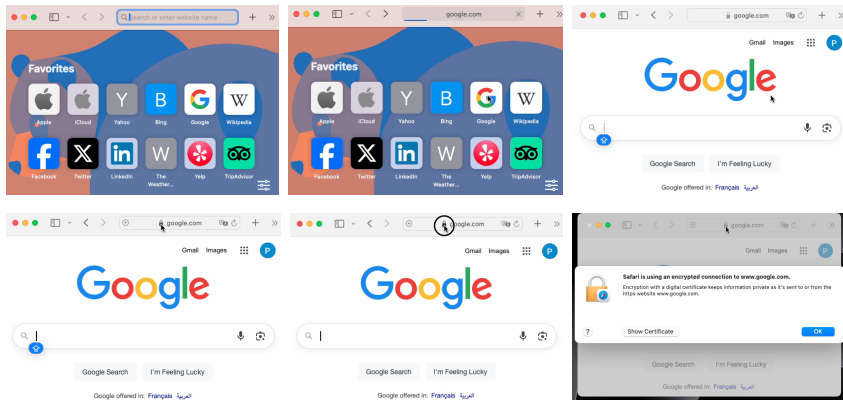
If two parties agree on a combination of cryptographic algorithms, they actually agree on a cipher suite. Other suites:

TLS_RSA_PSK_WITH_AES_128_CBC_SHA256

TLS_AES_128_GCM_SHA256

TLS Phases (From Teletubbies Perspective)

Typically, TLS establishes a secure connection as follows:

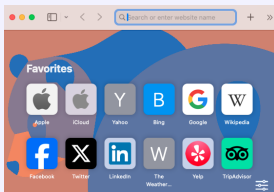


Actually, you just click on an HTTPS bound URL and let your web-browser do the work for you.

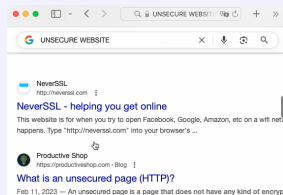
TLS Phases (From Teletubbies Perspective)

Typically, TLS establishes a secure connection as follows:

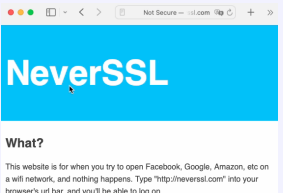
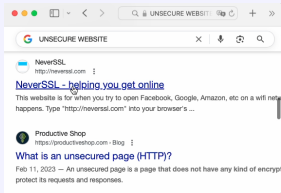
Of course, no one is stopping you from visiting unsecure websites through HTTP bound URLs:



then



then



Actual browser as the work for you.

TLS Phases (From Teletubbies Perspective)

Typically, TLS establishes a secure connection as follows:

You can test your domain TLS capabilities (online services):



TLS RESULTS FOR <https://tls-v1-0.badssl.com> [Copy test result URL](#) [New test →](#)

Moderate

TLS protocol versions ▾

TLS 1.3	Disabled
TLS 1.2	Enabled
TLS 1.1 (deprecated)	Enabled
TLS 1.0 (deprecated)	Enabled

TLS RESULTS FOR <https://ensia.edu.dz> [Copy test result URL](#) [New test →](#)

Excellent

TLS protocol versions ▾

TLS 1.3	Enabled
TLS 1.2	Enabled
TLS 1.1 (deprecated)	Enabled
TLS 1.0 (deprecated)	Enabled

TLS RESULTS FOR <https://neverssl.com> [Copy test result URL](#) [New test →](#)

Excellent

TLS protocol versions ▾

TLS 1.3	Enabled
TLS 1.2	Enabled
TLS 1.1 (deprecated)	Disabled
TLS 1.0 (deprecated)	Disabled

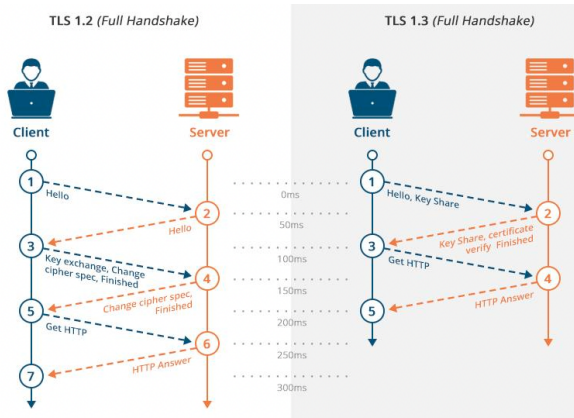
Actual browser do the work for you.

TLS Messages (Message not in TLS 1.3)

TLS uses different messages to establish and manage secure connections:

- ① **Handshake Messages.** Used to negotiate TLS session, authenticate parties, and establish encryption keys. This includes:
 - **Client Hello.** *Init, list supported TLS versions, cipher suites, etc.*
 - **Server Hello.** *Chooses TLS version, cipher suite, and extensions.*
 - **Certificate.** *Sends servers certificate.*
 - **ServerKeyExchange.** *Carries server's ephemeral parameters.*
 - **ClientKeyExchange.** *Carries server's ephemeral parameters.*
 - **CertificateRequest.** *Requests client certificate (optional).*
 - **ServerHelloDone.** *Server is done with handshake.*
 - **CertificateVerify.** *Contains signature of the handshake (use cert).*
 - **Finished.** *HMAC over all handshake messages (using derived key).*
- ② **Change Cipher Spec Message.** Signals that subsequent messages will be encrypted. Replaced by “**Encrypted Handshake Message**” in v1.3.
- ③ **Application Data.** Encrypted TLS data (HTTPS, SMTPS, etc).
- ④ **Alert Messages.** Signal errors. E.g., **Warning** are used for non-fatal issues and **Fatal** are used to force connection termination.

TLS Handshake (TLS 1.2 vs TLS 1.3)



TLS 1.3 differs drastically from SSL/TLS prior to TLS 1.3, where key exchange happened AFTER authentication. In SSL and TLS 1-1.2 certificates were sent as plaintext, whereas in TLS 1.3. they are encrypted.

TLS Phases (Abstractly)

Typically, TLS establishes a secure connection as follows:

TLS v1.2 and Earlier:

- ① **Client Hello.** Client shares with the server its capabilities along with some security information and parameters.
- ② **Server Hello.** Server chooses capabilities (supported by client) and shares security information and parameters with the client.
- ③ **Certificate.** Server shares its digital certificate with the client to verify its validity and server authenticity (prevent MITM scenarios).
- ④ **Key exchange and session key establishment.** Using the exchanged security parameters and a selected establishment protocol, both the client and the server compute the same session key.

In TLS 1.2, the certificate is sent in plaintext in a dedicated message. Also, the handshake requires from 7 to 9 messages. On average, it takes 300ms to complete the handshake. It supports the use of weak ciphers.

TLS Phases (Abstractly)

Typically, TLS establishes a secure connection as follows:

TLS v1.3:

- ① **Client Hello.** Client shares with the server its capabilities along with some security information and parameters.
- ② **Server Hello.** Server chooses capabilities (supported by client) and share security information and parameters with the client. It derives a key and encrypts its digital certificate and a confirmation message (to prove the correct derivation of the secret key).
- ③ **Authentication.** The client decrypts the certificate and verifies the identity of the server. Then, it confirms the correct derivation of the key. They both compute the same session key.

In TLS 1.3, the certificate is sent encrypted. Also, the handshake requires from 2 to 3 messages. On average, it takes 200ms to complete the handshake. It provide perfect **forward (& backward) secrecy**.

TLS 1.2 Phases (with more details)

TLS Phases — TLS 1.2

Typically, TLS establishes a secure connection through a three-phase process:

(1) Hello Phase. Runs as follows:

- **Client Hello:** Client starts the handshake with a “**Client Hello**” message indicating the supported TLS versions (1.0-1.2), cipher suites, and signature algorithms, the session ID, the server URL, and a random number.
- **Server Hello:** The server responds with a “**Server Hello**” indicating the selected TLS version and cipher suite, session ID, a random number.

(2) Server Authentication Phase (Optional). The server sends “**a certificate message**” containing its digital certificate (along with the related certificate chain — from issuer CA to the root CA). The client verifies the validity and authenticity of the server’s digital certificate (preventing MITM).

(3) Key Exchange and Session Establishment. Based on the cipher suite agreed upon & based on the TLS version, in this phase both parties compute the **same secret key** by applying a key agreement protocol (E.g., ECDHE — Viz., “**Key exchange messages**”). They confirm that by exchanging a message “**Finished**”, & then start exchanging encrypted messages (“**Application Data**”).

TLS Phases — TLS 1.2 (Wireshark)

This illustrates TLS handshake with the Algerian Ministry of National Defense's website (It uses TLSv1.2 — 05/2024) — [pcap file available](#):

No.	Time	Source	Destination	Protocol	Length	Info
8.66...	192.168.193.172	www.mdn.dz	192.168.193.172	TCP	78	54522 → https(443) [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=213466589 TSecr=0 SACK_PERM
8.78...	www.mdn.dz	192.168.193.172	192.168.193.172	TCP	74	https(443) → 54522 [SYN, ACK] Seq=0 Ack=1 Win=13600 Len=0 MSS=1460 WS=1 SACK_PERM TSval=24368395...
8.78...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=213466628 TSecr=2436839551
8.78...	192.168.193.172	www.mdn.dz	www.mdn.dz	TLSv1.2	583	Client Hello (SNI=www.mdn.dz)
8.74...	www.mdn.dz	192.168.193.172	192.168.193.172	TCP	66	https(443) → 54522 [ACK] Seq=1 Ack=518 Win=14117 Len=0 TSval=2436839596 TSecr=213466628
8.75...	www.mdn.dz	192.168.193.172	192.168.193.172	TLSv1.2	1414	Server Hello
8.75...	www.mdn.dz	192.168.193.172	192.168.193.172	TCP	1414	https(443) → 54522 [ACK] Seq=1349 Ack=518 Win=14117 Len=1348 TSval=2436839602 TSecr=213466628 [T...
8.75...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=518 Ack=1349 Win=130368 Len=0 TSval=213466681 TSecr=2436839602
8.75...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=518 Ack=2697 Win=129024 Len=0 TSval=213466681 TSecr=2436839602
8.75...	www.mdn.dz	192.168.193.172	192.168.193.172	TLSv1.2	1414	Certificate
8.75...	www.mdn.dz	192.168.193.172	192.168.193.172	TCP	118	https(443) → 54522 [PSH, ACK] Seq=4045 Ack=518 Win=14117 Len=52 TSval=2436839602 TSecr=213466628...
8.75...	www.mdn.dz	192.168.193.172	192.168.193.172	TLSv1.2	359	Server Key Exchange, Server Hello Done
8.75...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=518 Ack=4045 Win=129664 Len=0 TSval=213466682 TSecr=2436839602
8.75...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=518 Ack=4097 Win=129664 Len=0 TSval=213466682 TSecr=2436839602
8.75...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=518 Ack=4390 Win=130752 Len=0 TSval=213466682 TSecr=2436839604
8.76...	192.168.193.172	www.mdn.dz	www.mdn.dz	TLSv1.2	192	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
8.88...	www.mdn.dz	192.168.193.172	192.168.193.172	TCP	66	https(443) → 54522 [ACK] Seq=4390 Ack=644 Win=14243 Len=0 TSval=2436839649 TSecr=213466685
8.88...	www.mdn.dz	192.168.193.172	192.168.193.172	TLSv1.2	117	Change Cipher Spec, Encrypted Handshake Message
8.88...	192.168.193.172	www.mdn.dz	www.mdn.dz	TCP	66	54522 → https(443) [ACK] Seq=644 Ack=4441 Win=131008 Len=0 TSval=213466728 TSecr=2436839652
8.88...	192.168.193.172	www.mdn.dz	www.mdn.dz	TLSv1.2	529	Application Data
8.84...	www.mdn.dz	192.168.193.172	192.168.193.172	TCP	66	https(443) → 54522 [ACK] Seq=4441 Ack=1107 Win=14706 Len=0 TSval=2436839693 TSecr=213466728
8.86...	www.mdn.dz	192.168.193.172	192.168.193.172	TLSv1.2	1414	Application Data

For performance optimizations, TCP performs payload bundling of TLS messages. E.g., “Server Key Exchange” + “Server Hello Done” are bundled in one TCP segment. The message **Change Cipher Spec** signifies: the handshake is complete. **Application Data** contain encrypted content.

TLS Phases — TLS 1.2 (Wireshark)

This illustrates TLS handshake with the Center for Development of Advanced Technologies's website (It used TLSv1.2 — 05/2024):

No.	Time	Source	Destination	Protocol	Info
9.49...	192.168.193.172	www.cdta.dz	192.168.193.172	TCP	→ 59308 → https(443) [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=639789208 TSecr=0 SACK_PERM
9.53...	www.cdta.dz	192.168.193.172	192.168.193.172	TCP	→ https(443) → 59308 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1380 SACK_PERM TSval=3728754985 T
9.53...	192.168.193.172	www.cdta.dz	192.168.193.172	TCP	→ 59308 → https(443) [ACK] Seq=1 Ack=1 Win=131328 Len=0 TSval=639789252 TSecr=3728754985
9.53...	192.168.193.172	www.cdta.dz	192.168.193.172	TLSv1.2	Client Hello (SNI=www.cdta.dz)
9.60...	www.cdta.dz	192.168.193.172	192.168.193.172	TCP	→ https(443) → 59308 [ACK] Seq=1 Ack=564 Win=64640 Len=0 TSval=3728755039 TSecr=639789252
9.60...	www.cdta.dz	192.168.193.172	192.168.193.172	TLSv1.2	Server Hello
9.60...	www.cdta.dz	192.168.193.172	192.168.193.172	TLSv1.2	Certificate, Server Key Exchange, Server Hello Done
9.60...	192.168.193.172	www.cdta.dz	www.cdta.dz	TCP	→ 59308 → https(443) [ACK] Seq=564 Ack=1349 Win=129920 Len=0 TSval=639789322 TSecr=3728755042
9.60...	192.168.193.172	www.cdta.dz	www.cdta.dz	TCP	→ 59308 → https(443) [ACK] Seq=564 Ack=1995 Win=129280 Len=0 TSval=639789322 TSecr=3728755042
9.81...	192.168.193.172	www.cdta.dz	www.cdta.dz	TLSv1.2	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
9.81...	192.168.193.172	www.cdta.dz	www.cdta.dz	TLSv1.2	Application Data
9.85...	www.cdta.dz	192.168.193.172	192.168.193.172	TCP	→ https(443) → 59307 [ACK] Seq=1 Ack=1 Win=505 Len=0 TSval=3728755306 TSecr=112043810
9.85...	www.cdta.dz	192.168.193.172	192.168.193.172	TCP	→ https(443) → 59308 [ACK] Seq=1995 Ack=649 Win=64640 Len=0 TSval=3728755307 TSecr=639789530

For performance optimizations, TCP performs payload bundling of TLS messages. E.g., “Certificate message” + “Server Key Exchange” + “Server Hello Done” are bundled in one TCP segment.

The message **Change Cipher Spec** signifies: the handshake is complete, and the agreed-upon encryption algorithms and hashing functions become active.

Application Data contain encrypted content.

- **End.**